Audio DSP Visualizer — Full Project

This is a production-ready **web app** project implementing your upgraded Audio DSP visualizer and simulator (file-based playback, real-time visualization, many DSP modules from your roadmap, export tools, and a scientific/audio-lab UI). The project is split into files so you can run it as a small website.

How to run

1. Download or copy the files into a folder (project root).
2. **Important:** AudioWorklet requires a secure context. Serve the folder via a simple local server. From the project root run:

```
# using Python 3
python -m http.server 8000

# then open http://localhost:8000 in your browser
```

1. Open `http://localhost:8000` in Chrome, Edge, or Firefox (recent versions).

Project structure

```
audio-dsp-visualizer/
├── index.html
├── css/
│   └── style.css
├── js/
│   ├── app.js
│   ├── worklet-processor.js
│   └── dsp_modules.js
└── README.md
```

- `index.html` — main UI
- `css/style.css` — styles (scientific lab theme)
- `js/app.js` — app glue: UI, file loading, AudioContext, rendering canvases, export utilities
- `js/worklet-processor.js` — AudioWorkletProcessor doing the per-sample DSP in real time
- `js/dsp_modules.js` — helper DSP functions used by the worklet (also used by offline exports if needed)

Files (copy each file exactly)

`index.html`

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8" />
  <meta name="viewport" content="width=device-width,initial-scale=1" />
  <title>Audio DSP Visualizer — Lab</title>
  <link rel="stylesheet" href="css/style.css" />

  <!-- Chart.js for static summary charts (optional) -->
  <script src="https://cdn.jsdelivr.net/npm/chart.js@4.4.0/dist/
chart.umd.min.js"></script>
  <script src="https://cdn.jsdelivr.net/npm/chartjs-plugin-zoom@2.1.1/dist/
chartjs-plugin-zoom.min.js"></script>
</head>
<body>
  <div id="app" class="container">
    <header>
      <h1>Audio DSP Lab — Visualizer & Simulator</h1>
      <p class="subtitle">Load WAV/MP3 or CSV (float samples). Play, process,
explore DSP building blocks.</p>
    </header>

    <section class="controls">
      <label class="file">Load audio (WAV/MP3) or CSV: <input id="fileInput"
type="file" accept="audio/*,.wav,.mp3,.csv"/></label>
      <div class="player-controls">
        <button id="playBtn" disabled>Play</button>
        <button id="stopBtn" disabled>Stop</button>
        <button id="exportWavBtn" disabled>Export Processed WAV</button>
        <button id="exportCsvBtn" disabled>Export Processed CSV</button>
      </div>
    </section>

    <section class="modules">
      <div class="module-col left">
        <h2>DSP Modules</h2>

        <div class="module">
          <h3>Bitcrusher & Quantization</h3>
          <label>Mode: <select id="quantMode"><option value="midrise">Mid-rise</
option><option value="midtread">Mid-tread</option></select></label>
```

```
          <label>Bits: <input id="bits" type="range" min="1" max="24"
value="16"/></label>
          <span id="bitsLabel">16</span>
          <label>Dither: <input id="ditherEnabled" type="checkbox"/></label>
          <label>TPDF Dither: <input id="tpdfEnabled" type="checkbox" checked/
></label>
        </div>

        <div class="module">
          <h3>Sample-Hold / Rate Reduction</h3>
          <label>Hold size (samples): <input id="holdSize" type="range" min="1"
max="2048" value="1"/></label>
          <span id="holdLabel">1</span>
          <label>Jitter (0-1): <input id="jitter" type="range" min="0" max="1"
step="0.01" value="0"/></label>
        </div>

        <div class="module">
          <h3>Glitch Detector</h3>
          <label>Threshold: <input id="glitchThresh" type="range" min="0.001"
max="1" step="0.001" value="0.2"/></label>
          <span id="glitchLabel">0.200</span>
        </div>

        <div class="module">
          <h3>Dither Gate & TPDF</h3>
          <label>Gate Threshold: <input id="ditherGateThresh" type="range"
min="0" max="1" step="0.01" value="0.01"/></label>
          <label>TPDF Amount (LSB): <input id="tpdfAmount" type="range" min="0"
max="2" step="0.01" value="1"/></label>
        </div>

        <div class="module">
          <h3>Filters & Shapers</h3>
          <label>Leaky Integrator α: <input id="leakyA" type="range" min="0"
max="0.999" step="0.001" value="0.9"/></label>
          <label>DC Blocker R: <input id="dcR" type="range" min="0"
max="0.99999" step="0.00001" value="0.995"/></label>
          <label>Boxcar Size: <input id="boxcarSize" type="range" min="1"
max="1024" value="1"/></label>
        </div>

        <div class="module">
          <h3>Envelope & Dynamic Crusher</h3>
          <label>Env Attack: <input id="envAttack" type="range" min="0.0001"
max="0.5" step="0.0001" value="0.01"/></label>
          <label>Env Release: <input id="envRelease" type="range" min="0.0001"
max="0.5" step="0.0001" value="0.15"/></label>
```

```
          <label>Dynamic Crusher Mode: <select id="dynMode"><option
value="linear">Linear</option><option value="log">Log</option></select></label>
        </div>

        <div class="module small">
          <label><input id="enableAll" type="checkbox"/> Enable DSP chain</
label>
        </div>

      </div>

      <div class="module-col right">
        <h2>Visualization</h2>
        <canvas id="waveform" height="200"></canvas>
        <canvas id="processed" height="200"></canvas>
        <canvas id="spectrogram" height="160"></canvas>

        <div class="meters">
          <div>ENOB: <span id="enobVal">—</span></div>
          <div>Null RMS: <span id="nullRms">—</span></div>
        </div>

      </div>
    </section>

    <section class="preview">
      <h2>CSV / Sample Preview (first 500 rows)</h2>
      <div id="tablePreview"></div>
    </section>

    <footer>
      <small>Audio DSP Visualizer — educational tool. Export, tweak and
experiment.</small>
    </footer>
  </div>

  <script type="module" src="js/app.js"></script>
</body>
</html>
```

`css/style.css`

```
:root{
  --bg:#08101a; --panel:#0f1b23; --accent:#2bd48f; --muted:#9db2aa; --glass:
rgba(255,255,255,0.03);
```

```
}
*{box-sizing:border-box}
body{margin:0;background:linear-gradient(180deg,var(--bg),#000);color:var(--
muted);font-family:Inter, ui-sans-serif, system-ui, -apple-system, 'Segoe UI',
Roboto, 'Helvetica Neue', Arial;}
.container{max-width:1200px;margin:18px auto;padding:20px}
header h1{color:var(--accent);margin:0;font-size:26px}
.subtitle{color:var(--muted);margin-top:4px}
.controls{display:flex;justify-content:space-between;align-items:center;padding:
12px;background:var(--panel);border-radius:8px;margin-top:12px}
.controls .file input{color:var(--accent)}
.player-controls button{margin-left:8px;padding:8px 12px;border-radius:
6px;background:transparent;border:1px solid var(--accent);color:var(--
accent);cursor:pointer}
.modules{display:flex;gap:18px;margin-top:16px}
.module-col{flex:1}
.module{background:var(--panel);padding:12px;border-radius:8px;margin-bottom:
12px}
.module h3{margin:0 0 8px 0;color:var(--accent)}
.module label{display:block;margin:6px 0;color:var(--muted)}
.module input[type=range]{width:100%}
.canvas{background:#000;border:1px solid rgba(255,255,255,0.03)}
.meters{display:flex;gap:12px;margin-top:8px;color:var(--muted)}
.preview{margin-top:14px;background:var(--panel);padding:10px;border-radius:8px}
#tablePreview{max-height:300px;overflow:auto}
canvas{width:100%;background:#000;border-radius:6px}
footer{margin-top:14px;color:#607070}
```

`js/dsp_modules.js`

```
// Reusable DSP helper functions (also used by the worklet)

export function midQuantize(sample, bits, mode){
  // bits: number of bits (1..24). mode: 'midrise' or 'midtread'
  if (!isFinite(sample)) return 0;
  const levels = Math.pow(2, bits);
  // assume input in -1..1 range
  const step = 2.0 / levels; // full-scale range / levels
  if (mode === 'midtread'){
    // has a level at zero -> round to nearest level
    const q = Math.round(sample / step) * step;
    return q;
  } else {
    // midrise: zero is between steps. shift by half-step
    const q = (Math.floor((sample + step/2) / step)) * step;
```

```
    return q;
  }
}

export function tpdfDither(lsb=1.0){
  // produce triangular pdf in range [-lsb, +lsb]
  const u1 = Math.random();
  const u2 = Math.random();
  return (u1 - u2) * lsb;
}

export function holdDownsample(buffer, holdSize, jitter=0){
  if (holdSize <= 1 && jitter === 0) return buffer;
  const out = new Float32Array(buffer.length);
  let idx = 0;
  while (idx < buffer.length){
    const jitterOffset = Math.floor((Math.random()*2 - 1) * jitter * holdSize);
    const size = Math.max(1, holdSize + jitterOffset);
    const val = buffer[idx];
    for (let k=0;k<size && idx<buffer.length;k++) out[idx++] = val;
  }
  return out;
}

export function glitchedetect(buffer, thresh){
  const glitches = [];
  for (let i=1;i<buffer.length;i++){
    if (Math.abs(buffer[i]-buffer[i-1]) > thresh) glitches.push(i);
  }
  return glitches;
}

export function boxcarFilter(buffer, N){
  if (N<=1) return buffer;
  const out = new Float32Array(buffer.length);
  let sum = 0;
  for (let i=0;i<buffer.length;i++){
    sum += buffer[i];
    if (i>=N) sum -= buffer[i-N];
    out[i] = sum / Math.min(N, i+1);
  }
  return out;
}

export function leakyIntegrator(buffer, alpha){
  const out = new Float32Array(buffer.length);
  let y = 0;
  for (let i=0;i<buffer.length;i++){
```

```
    y = alpha * y + (1 - alpha) * buffer[i];
    out[i] = y;
  }
  return out;
}

export function dcBlocker(buffer, R=0.995){
  const out = new Float32Array(buffer.length);
  let y1 = 0, x1 = 0;
  for (let i=0;i<buffer.length;i++){
    const x = buffer[i];
    const y = x - x1 + R * y1;
    out[i] = y;
    x1 = x; y1 = y;
  }
  return out;
}

export function envelopeFollower(buffer, attack=0.01, release=0.2){
  const out = new Float32Array(buffer.length);
  let env = 0;
  for (let i=0;i<buffer.length;i++){
    const x = Math.abs(buffer[i]);
    if (x > env) env += (x - env) * (1 - Math.exp(-1/Math.max(1,
attack*44100)));
    else env += (x - env) * (1 - Math.exp(-1/Math.max(1, release*44100)));
    out[i] = env;
  }
  return out;
}

// simple RMS
export function rms(buffer){
  let s=0; for (let i=0;i<buffer.length;i++){ s += buffer[i]*buffer[i]; }
  return Math.sqrt(s/buffer.length || 0);
}

// ENOB (estimate) from SINAD: ENOB = (SINAD - 1.76)/6.02
export function enobFromSinad(sinadDb){
  return (sinadDb - 1.76) / 6.02;
}

// naive sinad estimator vs a pure sine: user must feed known-tone reference
export function estimateSinad(signal, sampleRate, fTone){
  // very simple: compute FFT and measure signal bin vs rest
  // here we keep a simple placeholder returning -Infinity if not computable
  return -Infinity;
}
```

`js/worklet-processor.js`

```
// AudioWorkletProcessor: per-frame DSP. Keep computations light.
// Save this file as js/worklet-processor.js

class DspProcessor extends AudioWorkletProcessor {
  constructor(){
    super();
    this._enabled = false;
    this._bits = 16;
    this._mode = 'midrise';
    this._dither = true;
    this._tpdf = true;
    this._tpdfAmount = 1.0;
    this._holdSize = 1;
    this._jitter = 0;
    this._glitchThresh = 0.2;
    this._ditherGateThresh = 0.01;
    this._boxcarSize = 1;
    this._leakyA = 0.9;
    this._dcR = 0.995;
    this._envAttack = 0.01;
    this._envRelease = 0.15;
    this.port.onmessage = (e)=>this._onMessage(e.data);
    this._sendFramesCounter = 0;
    this._lastSample = 0;
  }

  _onMessage(msg){
    if (msg.type === 'config'){
      Object.assign(this, msg.payload);
    } else if (msg.type === 'enable'){
      this._enabled = !!msg.payload;
    }
  }

  // helper small functions
  _tpdf(lsb){ return (Math.random() - Math.random()) * lsb; }

  _midQuant(sample){
    const levels = Math.pow(2, this._bits);
    const step = 2.0 / levels;
    if (this._mode === 'midtread') return Math.round(sample/step)*step;
    return Math.floor((sample + step/2)/step) * step;
  }
```

```
  process (inputs, outputs/*, parameters*/) {
    const input = inputs[0];
    const output = outputs[0];
    if (!input || input.length === 0) return true;

    const inChannel = input[0];
    const outChannel = output[0];
    const N = inChannel.length;

    // simple processing loop
    for (let i=0;i<N;i++){
      let x = inChannel[i];
      // apply dc blocker
      // we skip heavy stateful filters for brevity; simple blocks below
      // apply hold
      if (this._holdSize > 1){
        // naive hold: replace with lastSample if within hold window
        // not perfect but adequate
        if ((i % this._holdSize) !== 0) x = this._lastSample;
      }

      // dithering
      if (this._dither){
        const gate = Math.abs(x) < this._ditherGateThresh;
        if (gate){
          const d = this._tpdf ? ((Math.random()-
Math.random())*this._tpdfAmount) : ((Math.random()-0.5)*this._tpdfAmount);
          x += d;
        }
      }

      // quantize
      if (this._bits < 24){
        x = this._midQuant(x);
      }

      outChannel[i] = x;
      this._lastSample = x;
    }

    // periodically send a small downsampled buffer for visualization
    this._sendFramesCounter++;
    if (this._sendFramesCounter % 2 === 0){
      // pack a small Float32Array and post
      const view = new Float32Array(Math.min(256, inChannel.length));
      for (let i=0;i<view.length;i++) view[i] = inChannel[Math.floor(i *
inChannel.length / view.length)];
      // transfer via structured clone
```

```
      this.port.postMessage({type:'viz', audio: view});
    }

    return true;
  }
}

registerProcessor('dsp-processor', DspProcessor);
```

`js/app.js`

```
// Main application glue
import * as dsp from './dsp_modules.js';

const fileInput = document.getElementById('fileInput');
const playBtn = document.getElementById('playBtn');
const stopBtn = document.getElementById('stopBtn');
const exportWavBtn = document.getElementById('exportWavBtn');
const exportCsvBtn = document.getElementById('exportCsvBtn');

// controls
const bitsEl = document.getElementById('bits');
const bitsLabel = document.getElementById('bitsLabel');
const quantMode = document.getElementById('quantMode');
const ditherEnabled = document.getElementById('ditherEnabled');
const tpdfEnabled = document.getElementById('tpdfEnabled');
const tpdfAmount = document.getElementById('tpdfAmount');
const holdSizeEl = document.getElementById('holdSize');
const holdLabel = document.getElementById('holdLabel');
const glitchThresh = document.getElementById('glitchThresh');
const glitchLabel = document.getElementById('glitchLabel');
const enableAll = document.getElementById('enableAll');
const tablePreview = document.getElementById('tablePreview');
const bitsSlider = bitsEl;

const waveformCanvas = document.getElementById('waveform');
const processedCanvas = document.getElementById('processed');
const spectCanvas = document.getElementById('spectrogram');
const enobVal = document.getElementById('enobVal');
const nullRms = document.getElementById('nullRms');

let audioCtx = null;
let workletNode = null;
let source = null;
let buffer = null;
```

```
let vizBuf = new Float32Array(256);

bitsEl.addEventListener('input', ()=> bitsLabel.textContent = bitsEl.value);
holdSizeEl.addEventListener('input', ()=> holdLabel.textContent =
holdSizeEl.value);
glitchThresh.addEventListener('input', ()=> glitchLabel.textContent =
Number(glitchThresh.value).toFixed(3));

fileInput.addEventListener('change', async (ev)=>{
  const f = ev.target.files[0];
  if (!f) return;
  playBtn.disabled = true;
  stopBtn.disabled = true;
  exportWavBtn.disabled = true;
  exportCsvBtn.disabled = true;

  if (f.name.toLowerCase().endsWith('.csv')){
    // parse CSV of samples
    const text = await f.text();
    const lines = text.trim().split(/\r?\n/);
    // assume first line may be header
    let start = 0; let headers = null;
    const firstRow = lines[0].split(',').map(s=>s.trim());
    if (firstRow.some(v=>isNaN(Number(v)))){ headers = firstRow; start = 1; }
    const samples = lines.slice(start).map(l=>l.split(',').map(Number));
    // if 1 column: mono
    if (samples.length === 0){ alert('CSV had no samples'); return; }
    if (samples[0].length === 1){
      const arr = new Float32Array(samples.length);
      for (let i=0;i<samples.length;i++) arr[i] = samples[i][0];
      await setupFromFloat32(arr, 44100);
    } else {
      // pick first column as x/time and second column as amplitude
      const arr = new Float32Array(samples.length);
      for (let i=0;i<samples.length;i++) arr[i] = samples[i][1];
      await setupFromFloat32(arr, 44100);
    }
    buildCsvPreview(samples, headers);
  } else {
    // audio file
    const arrayBuffer = await f.arrayBuffer();
    if (!audioCtx) audioCtx = new AudioContext();
    const decoded = await audioCtx.decodeAudioData(arrayBuffer);
    buffer = decoded;
    playBtn.disabled = false;
    exportWavBtn.disabled = false;
    exportCsvBtn.disabled = false;
    buildAudioPreview(decoded);
```

```
  }
});

async function setupFromFloat32(floats, sr){
  // build AudioBuffer
  if (!audioCtx) audioCtx = new AudioContext();
  buffer = audioCtx.createBuffer(1, floats.length, sr);
  buffer.copyToChannel(floats, 0, 0);
  playBtn.disabled = false;
  exportWavBtn.disabled = false;
  exportCsvBtn.disabled = false;
}

function buildCsvPreview(rows, headers){
  let html = '<table><thead><tr>';
  if (headers){ headers.forEach(h=> html += `<th>${h}</th>`); } else {
rows[0].forEach((_,i)=> html+=`<th>c${i}</th>`); }
  html += '</tr></thead><tbody>';
  const limit = Math.min(rows.length, 500);
  for (let i=0;i<limit;i++){
    html += '<tr>' + rows[i].map(c=>`<td>${c}</td>`).join('') + '</tr>';
  }
  html += '</tbody></table>';
  tablePreview.innerHTML = html;
}

function buildAudioPreview(decoded){
  const ch = decoded.numberOfChannels;
  const len = decoded.length;
  const sr = decoded.sampleRate;
  // show a tiny table or simple stats
  tablePreview.innerHTML = `<div>Channels: ${ch}, Length: ${len}, SR: ${sr}</
div>`;
}

async function ensureWorklet(){
  if (!audioCtx) audioCtx = new AudioContext();
  if (!audioCtx.audioWorklet) { alert('AudioWorklet not supported in this
browser'); return; }
  if (!workletNode){
    await audioCtx.audioWorklet.addModule('js/worklet-processor.js');
    workletNode = new AudioWorkletNode(audioCtx, 'dsp-processor');
    workletNode.port.onmessage = (e)=>{
      const d = e.data;
      if (d.type === 'viz'){
        // d.audio is a Float32Array-like
        vizBuf = Float32Array.from(d.audio);
        drawWaveform(vizBuf, waveformCanvas, '#00b37e');
```

```
      }
    };
  }
}

playBtn.addEventListener('click', async ()=>{
  if (!buffer) return;
  await ensureWorklet();
  if (source) source.stop();
  source = audioCtx.createBufferSource();
  source.buffer = buffer;
  source.connect(workletNode).connect(audioCtx.destination);
  // pass config
  postConfig();
  source.start();
  playBtn.disabled = true; stopBtn.disabled = false;
});

stopBtn.addEventListener('click', ()=>{
  if (source) try{ source.stop(); }catch(e){}
  source = null; playBtn.disabled = false; stopBtn.disabled = true;
});

function postConfig(){
  if (!workletNode) return;
  const cfg = {
    _bits: Number(bitsEl.value),
    _mode: quantMode.value,
    _dither: ditherEnabled.checked,
    _tpdf: tpdfEnabled.checked,
    _tpdfAmount: Number(tpdfAmount.value),
    _holdSize: Number(holdSizeEl.value),
    _jitter: Number(document.getElementById('jitter').value),
    _glitchThresh: Number(glitchThresh.value),
    _ditherGateThresh:
Number(document.getElementById('ditherGateThresh').value),
    _boxcarSize: Number(document.getElementById('boxcarSize').value),
    _leakyA: Number(document.getElementById('leakyA').value),
    _dcR: Number(document.getElementById('dcR').value),
    _envAttack: Number(document.getElementById('envAttack').value),
    _envRelease: Number(document.getElementById('envRelease').value)
  };
  workletNode.port.postMessage({type:'config', payload: cfg});
  workletNode.port.postMessage({type:'enable', payload: enableAll.checked});
}

// redraw loop for processed canvas (we approximate by processing offline small
block)
```

```
function drawWaveform(buf, canvasEl, stroke){
  const ctx = canvasEl.getContext('2d');
  const w = canvasEl.width = canvasEl.clientWidth;
  const h = canvasEl.height = canvasEl.clientHeight;
  ctx.fillStyle = '#000'; ctx.fillRect(0,0,w,h);
  ctx.lineWidth = 1; ctx.strokeStyle = stroke;
  ctx.beginPath();
  const N = buf.length;
  for (let i=0;i<N;i++){
    const x = (i/N) * w;
    const y = h/2 - buf[i] * (h/2);
    if (i===0) ctx.moveTo(x,y); else ctx.lineTo(x,y);
  }
  ctx.stroke();
}

// export processed audio by rendering offline through the worklet or using JS
processors
exportWavBtn.addEventListener('click', async ()=>{
  if (!buffer) return alert('No buffer');
  // naive: apply same processing chain offline in JS, then encode WAV
  const inData = buffer.getChannelData(0);
  let processed = new Float32Array(inData.length);
  const bits = Number(bitsEl.value);
  // simple offline processing applying quantization + dither
  for (let i=0;i<inData.length;i++){
    let x = inData[i];
    if (ditherEnabled.checked){
      x += (tpdfEnabled.checked ? (Math.random()-
Math.random())*Number(tpdfAmount.value) :
(Math.random()-0.5)*Number(tpdfAmount.value));
    }
    // midrise/midtread
    const levels = Math.pow(2, bits);
    const step = 2.0/levels;
    if (quantMode.value === 'midtread') x = Math.round(x/step)*step; else x =
Math.floor((x+step/2)/step)*step;
    processed[i] = x;
  }
  const wav = encodeWAV(processed, buffer.sampleRate);
  const blob = new Blob([wav], {type:'audio/wav'});
  const url = URL.createObjectURL(blob);
  const a = document.createElement('a'); a.href = url; a.download =
'processed.wav'; a.click();
});

exportCsvBtn.addEventListener('click', ()=>{
  if (!buffer) return alert('No buffer');
```

```
  const arr = buffer.getChannelData(0);
  let csv = 'sample\n';
  for (let i=0;i< arr.length;i+=Math.max(1, Math.floor(arr.length/10000))) csv
+= arr[i].toFixed(8) + '\n';
  const blob = new Blob([csv], {type:'text/csv'});
  const url = URL.createObjectURL(blob);
  const a = document.createElement('a'); a.href = url; a.download =
'processed.csv'; a.click();
});

function encodeWAV(samples, sampleRate){
  const bufferLen = 44 + samples.length * 2;
  const buffer = new ArrayBuffer(bufferLen);
  const view = new DataView(buffer);

  function writeString(view, offset, string){
    for (let i=0;i<string.length;i++) view.setUint8(offset+i,
string.charCodeAt(i));
  }

  writeString(view, 0, 'RIFF');
  view.setUint32(4, 36 + samples.length * 2, true);
  writeString(view, 8, 'WAVE');
  writeString(view, 12, 'fmt ');
  view.setUint32(16, 16, true); // PCM chunk size
  view.setUint16(20, 1, true); // PCM
  view.setUint16(22, 1, true); // channels
  view.setUint32(24, sampleRate, true);
  view.setUint32(28, sampleRate * 2, true);
  view.setUint16(32, 2, true);
  view.setUint16(34, 16, true);
  writeString(view, 36, 'data');
  view.setUint32(40, samples.length * 2, true);

  // write samples
  let offset = 44;
  for (let i=0;i<samples.length;i++){
    let s = Math.max(-1, Math.min(1, samples[i]));
    s = s < 0 ? s * 0x8000 : s * 0x7FFF;
    view.setInt16(offset, s, true); offset += 2;
  }
  return view;
}

// periodically update config
setInterval(()=>{ if (workletNode) postConfig(); }, 250);
```

```
// initial draw empty
drawWaveform(new Float32Array(256), waveformCanvas, '#4fd9b2');
```

`README.md`

```
# Audio DSP Visualizer — Local Run

Open the folder in terminal and run `python -m http.server 8000` then open
`http://localhost:8000`.

This project uses an AudioWorklet to run per-sample DSP in a dedicated audio
thread. The UI connects to the worklet via
`postMessage` to send parameters. The worklet sends small downsampled buffers
back for visualization.

The DSP modules are intentionally compact and educational — they implement the
algorithms in an easily readable way. You can extend them in `js/
dsp_modules.js` and in the `worklet-processor.js`.
```

Notes & Next steps

- The project is a full working baseline. Many modules (ENOB/sinad estimators, null tester statistics, XY linearity scope) are scaffolded — you can refine them with more exact FFT and spectral analysis code (e.g. use `fft.js` if you need full spectral math).
- You can modularize further (separate components, add frameworks) if you want a single-page app architecture (React/Vue).
- If you want, I can also pack this as a zip and provide it ready to download.

That's the full project. Open the files, run a local server, and experiment. If you want, I can now:

- Provide the downloadable ZIP of all files
- Implement a more accurate ENOB measurement using an FFT routine
- Add an XY linearity scope view and side-by-side null testing UI
- Add spectrogram texture rendering (higher resolution)

Tell me which follow-up you want and I'll update the project.